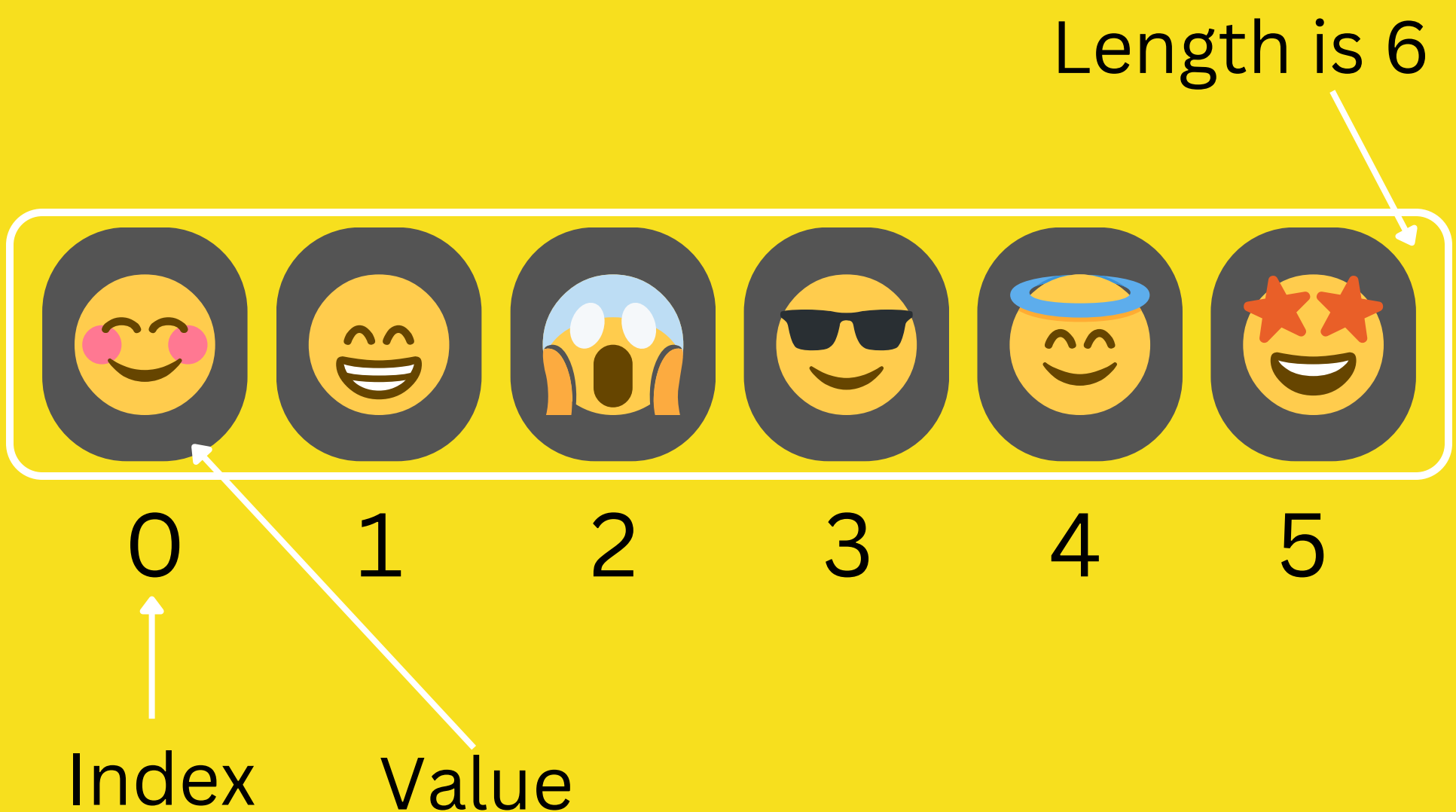


Top 20 JavaScript Array Methods

Everyone Must Know



.push()

- **What it does:** Adds elements to the end of an array.
- **Use case:** When you need to expand your array.

```
let emotions = ["😊", "😱"];  
emotions.push("😁"); // Adds "😁" to the end  
console.log(emotions); // ["😊", "😱", "😁"]
```



.pop()

- **What it does:** Removes the last element from an array.
- **Use case:** When you need to remove and retrieve the last item.

```
let emotions = ["😊", "😱"];  
let lastEmotion = emotions.pop(); // Removes "😱"  
console.log(emotions); // ["😊"]  
console.log(lastEmotion); // "😱"
```



.shift()

- **What it does:** Removes the first element from an array.
- **Use case:** When you need to remove and retrieve the first item.

```
let emotions = ["😊", "😱"];  
let firstEmotion = emotions.shift(); // Removes "😊"  
console.log(emotions); // ["😱"]  
console.log(firstEmotion); // "😊"
```



.unshift()

- **What it does:** Adds elements to the beginning of an array.
- **Use case:** When you need to prepend data.

```
let emotions = ["😱"];  
emotions.unshift("😊"); // Adds "😊" at the beginning  
console.log(emotions); // ["😊", "😱"]
```



.map()

- **What it does:** Creates a new array by applying a function to each element.
- **Use case:** When you need to transform an array without modifying the original.

```
let numbers = [1, 2, 3];  
let doubled = numbers.map(num => num * 2);  
console.log(doubled); // [2, 4, 6]
```



.filter()

- **What it does:** Creates a new array with elements that pass a condition.
- **Use case:** When you need to filter elements based on a criterion.

```
let numbers = [1, 2, 3, 4];  
let evens = numbers.filter(num => num % 2 === 0);  
console.log(evens); // [2, 4]
```



.reduce()

- **What it does:** Reduces an array to a single value.
- **Use case:** When you need to sum or aggregate data.

```
let numbers = [1, 2, 3, 4];  
let sum = numbers.reduce((acc, num) => acc + num, 0);  
console.log(sum); // 10
```



.forEach()

- **What it does:** Iterates over an array without returning a new one.
- **Use case:** When you need to perform an action on each item.

```
let emotions = ["😊", "😱"];  
emotions.forEach(emotion => console.log(emotion));
```



.find()

- **What it does:** Returns the first element that matches a condition.
- **Use case:** When you need to find a specific item in an array.

```
let numbers = [1, 2, 3, 4];  
let firstEven = numbers.find(num => num % 2 === 0);  
console.log(firstEven); // 2
```



.findIndex()

- **What it does:** Returns the index of the first matching element.
- **Use case:** When you need the position of an item, not the value.

```
let numbers = [1, 2, 3, 4];  
let index = numbers.findIndex(num => num % 2 === 0);  
console.log(index); // 1
```



.includes()

- **What it does:** Checks if an element exists in an array.
- **Use case:** When you need to verify if a value is present.

```
let emotions = ["😊", "😱"];  
console.log(emotions.includes("😊")); // true
```



.some()

- **What it does:** Checks if at least one element meets a condition.
- **Use case:** When you need to know if any item passes a test.

```
let numbers = [1, 2, 3, 4];  
console.log(numbers.some(num => num > 3)); // true
```



.every()

- **What it does:** Checks if all elements meet a condition.
- **Use case:** When you need to ensure all items pass a test.

```
let numbers = [2, 4, 6];  
console.log(numbers.every(num => num % 2 === 0)); // true
```



.sort()

- **What it does:** Sorts an array in place.
- **Use case:** When you need to order elements.



```
let numbers = [3, 1, 4, 2];  
numbers.sort();  
console.log(numbers); // [1, 2, 3, 4]
```



.reverse()

- **What it does:** Reverses the order of elements.
- **Use case:** When you need to invert an array.



```
let numbers = [1, 2, 3];  
numbers.reverse();  
console.log(numbers); // [3, 2, 1]
```



.concat()

- **What it does:** Merges multiple arrays.
- **Use case:** When you need to combine lists.

```
let a = [1, 2];  
let b = [3, 4];  
let combined = a.concat(b);  
console.log(combined); // [1, 2, 3, 4]
```



.slice()

- **What it does:** Returns a portion of an array.
- **Use case:** When you need a specific range of elements.

```
let numbers = [1, 2, 3, 4];  
let sliced = numbers.slice(1, 3);  
console.log(sliced); // [2, 3]
```



.splice()

- **What it does:** Adds or removes elements at a specified index.
- **Use case:** When you need to modify an array in place.



```
let numbers = [1, 2, 3, 4];  
numbers.splice(1, 2, 99);  
console.log(numbers); // [1, 99, 4]
```



.indexOf()

- **What it does:** Returns the index of an element.
- **Use case:** When you need to locate an item in an array.

```
let emotions = ["😊", "😱"];  
console.log(emotions.indexOf("😱")); // 1
```



.flat()

- **What it does:** Flattens nested arrays into a single-level array.
- **Use case:** When you need to work with deeply nested data.

```
let nested = [1, [2, [3, 4]]];  
console.log(nested.flat(2)); // [1, 2, 3, 4]
```

